DW-Bench: Benchmarking LLMs on Data Warehouse Graph Topology Reasoning

Ahmed G.A.H Ahmed
Innosol
ahmed.gamal@innosol.online
Okan Saka
Bahçeşehir University
okansakars@gmail.com

Abstract

Text-to-SQL benchmarks test whether a model can write the right query. They do not test whether it understands the schema it queries. A data warehouse schema is a graph: tables linked by foreign keys, rows anchored to source records through ETL lineage. Tracing that structure (finding join paths, detecting disconnected silos, propagating impact through lineage chains) is routine data engineering, but no existing benchmark measures it.

We present **DW-Bench**, a benchmark for graph topology reasoning over data warehouse schemas. The schema-level split contains 1,046 questions (13 subtypes, three difficulty levels, five datasets, 262 tables). A separate value-level split adds 953 provenance questions that require interactive row-level data access, a regime where context-window approaches cannot compete. We run six baselines, from flat context injection to tool-calling and code execution, against Gemini 2.5 Flash, DeepSeek-V3, and Qwen2.5-72B.

Tool-augmented baselines reach 87–90% micro-EM; static methods top out at 63–81%. The gap narrows on easy subtypes and widens on hard ones: no method exceeds 61% while the oracle clears $\geq$99.5%. That gap is a reasoning problem. On value-level questions, tools flip `cascade_count` from 0% to an average of 75%, but `row_impact` stays at 16–17% on real-world schemas. Tool-use on real-world lineage outperforms the synthetic split because standard ETL patterns align with the tool API, whereas the synthetic schema was designed to maximize structural difficulty. Obfuscating table names drops static baselines 9–32 pp; tool-use loses under 4 pp.

Code, data, and per-question results: `https://github.com/AJamal27891/dw-bench`.

1 Introduction

Text-to-SQL has made rapid progress: state-of-the-art LLMs now approach or match human-level accuracy on Spider [21], BIRD [13], and Spider 2.0 [11], all of which measure a model's capacity to translate a natural-language question into a SQL query.

Data warehouses, on the other hand, present a different kind of problem. A data engineer maintaining an enterprise warehouse does not primarily write SQL. Instead, they trace graph structure: lineage edges, FK paths, disconnected schema silos. Common questions include:

- *"If the `Person` table changes, which downstream tables are affected through data lineage?"* (impact analysis)
- *"Which tables form disconnected silos in our schema graph?"* (connectivity analysis)
- *"What is the shortest FK path between two tables?"* (routing)

Answering these questions demands BFS, connected-component detection, and multi-hop path enumeration. No existing benchmark assesses LLMs on any of these capabilities.

Contributions.

1. **DW-Bench**: 1,046 schema-level and 953 value-level questions across 5 data warehouse schemas (4 real-world, 1 synthetic), covering 3 categories (lineage impact, schema routing, silo detection), 13 subtypes, and 3 difficulty levels.
2. **Six baselines** (Flat Text, Vector-RAG, Graph-Augmented, Tool-Use, ReAct-Code, Oracle), tested with Gemini 2.5 Flash, DeepSeek-V3, and Qwen2.5-72B.
3. **An obfuscation protocol** that randomizes table names, enabling controlled evaluation of memorization versus genuine topology understanding.
4. **Syn-Logistics**: A synthetic dataset with $n \geq 20$ per subtype. It exposes a model-dependent structural gap: Gemini scores 93–100% on `hop_count/count`, while DeepSeek drops to 27–48% on the same subtypes.
5. **Open-source release** of all datasets, evaluation code, and baseline implementations.

2 Related Work

Text-to-SQL Benchmarks. Spider [21] set the template: 10,181 questions across 200 databases. BIRD [13] introduced noisy schemas; Spider 2.0 [11] scaled to enterprise schemas. ScienceBenchmark targets scientific databases. RobuT [23] probes cell/row logic. All of these test *query generation or cell reasoning*, none test structural understanding.

Schema Understanding. BEAVER [1] evaluates LLMs on database knowledge including schema matching and entity resolution, but does not evaluate graph topology reasoning. LLM-FK [18] evaluates FK discovery, a single-hop structural task complementary to DW-Bench's multi-hop topology reasoning.

Graph Reasoning in LLMs. NLGraph [20] and GraphQA [4] test LLMs on synthetic graph problems (connectivity, shortest path). GraCoRe [22] taxonomizes graph reasoning; GraphArena [17] benchmarks graph algorithms at scale. None of these use real database schemas, and none mix FK with lineage edge types.

Graph-Centric Agents and GNN Hybrids. Think-on-Graph [16] interleaves LLM reasoning with KG exploration; Graph Chain-of-Thought [9] augments prompts with graph-derived reasoning chains. G-Retriever [7] uses GNN encoders to retrieve graph context for LLM QA. Retrieval-augmented generation [12] and hierarchical GraphRAG [3] address knowledge-intensive tasks but do not specifically target schema topology. DW-Bench does not compare against these advanced graph-agent frameworks; following the precedent of Spider and BIRD, we evaluate standardized, zero-shot baselines to establish a *lower bound*. These methods are natural next steps: the Oracle upper bound ($\geq$99.5% EM) quantifies exactly how much room remains for hybrid architectures combining tools with learned graph representations.

Positioning. DW-Bench uniquely combines: (1) *real-world* schemas from industry standards, (2) *heterogeneous* edge types (FK + lineage), (3) *practical* question types from data engineering workflows, and (4) *obfuscation* for contamination control.

3 DW-Bench: Benchmark Design

3.1 Datasets

We select five datasets representing diverse data warehouse topologies:

Table 1: DW-Bench dataset statistics. "Silos" = disconnected components in the combined FK+lineage graph.

Dataset	Domain	Tbl	FK	Lin.	Qs	Silos
AdventureWorks	Retail	102	136	39	208	11
TPC-DS	Analytics	24	70	0	127	1
TPC-DI	ETL	35	29	21	181	2
OMOP CDM	Health	37	74	21	158	3
Syn-Logistics	Supply	64	96	35	372	5
Total		**262**	**405**	**116**	**1046**	—

AdventureWorks is a Microsoft reference data warehouse with 102 tables spanning OLTP and DW layers, connected by 136 FK edges and 39 lineage (derived_from) edges. Its dual-layer structure makes it ideal for lineage impact questions.

TPC-DS is the industry standard analytics benchmark with a star schema of 24 tables and 70 FK edges. Its single connected component and absence of lineage edges tests pure FK-based reasoning.

TPC-DI is the data integration benchmark with 35 tables modeling an ETL pipeline from staging to warehouse, with 21 lineage edges representing the transformation flow.

OMOP CDM (Observational Medical Outcomes Partnership Common Data Model) [8] is a healthcare standard with 37 tables, 74 FK edges, and 21 lineage edges across 3 connected components (clinical data, vocabulary tables, and metadata).

Syn-Logistics is our own synthetic supply-chain schema: 64 tables, 5 connected components (carrier, procurement, healthcare, finance, HR). We built it to fix a statistical-power problem in the real-world corpus: every subtype gets $n \geq 20$ questions. Table names come from a domain dictionary so obfuscation does not collapse semantics.

3.2 Schema Graph Representation

Each dataset is represented as a heterogeneous graph $G = (V, E_{fk} \cup E_{lin})$ using PyTorch Geometric [5] `HeteroData`. Nodes represent tables with 6 structural features: in-degree, out-degree, normalized degree, lineage degree, betweenness centrality, and PageRank. Edges are typed as either `fk_to` (foreign key) or `derived_from` (data lineage). Lineage edges form a strict DAG (directed acyclic graph): each `derived_from` edge points from a downstream DW table to its upstream source, with no cycles or self-references. All traversal algorithms respect edge typing: FK queries use only `fk_to` edges, lineage queries use only `derived_from`, and `combined_impact` queries compose both types sequentially. For path-based questions, we validate alternative valid shortest paths against the FK adjacency matrix, ensuring EM is robust to tie-breaking.

3.3 Question Generation

Questions are generated deterministically from the graph structure using standard graph algorithms (BFS, Dijkstra, connected component detection) implemented with NetworkX [6], ensuring ground-truth answers are provably correct. We define three categories with 13 subtypes:

Difficulty assignment. Difficulty labels are assigned a priori based on structural complexity, following the precedent of Spider [21] (which uses SQL AST depth) and BIRD [13]. Easy subtypes require a single graph lookup (one hop, one edge type). Medium subtypes require multi-hop traversal within a single edge type. Hard subtypes require either multi-hop transitive closure or composition across both FK and lineage edge types. These labels are fixed properties of the question structure, independent of any model's empirical performance.

Table 2: Question taxonomy. Difficulty is assigned based on the number of reasoning hops and edge types required.

Category	Subtype	Diff.	Description	#Qs
Lin.	forward	Easy	Direct lineage targets	72
	reverse	Easy	Source tables for DW table	71
	transitive	Hard	Multi-hop lineage chains	33
	combined_impact	Hard	Lineage + FK dependents	69
	multi_source	Med.	Tables with 3+ sources	49
Rt.	direct_fk	Easy	FK adjacency check	100
	join_path	Med.	FK shortest path	342
	hop_count	Med.	Path length	124
Silo	count	Easy	Number of components	30
	membership	Hard	Which component has X?	38
	isolation	Med.	Is table X isolated?	60
	connected	Med.	Are X and Y connected?	29
	full_enum.	Hard	List all tables in silo	29

3.4 Obfuscation Protocol

To distinguish genuine topology reasoning from schema name memorization, we generate an *obfuscated* variant for each dataset. Table names are replaced with random identifiers (`Table_A`, `Table_B`, etc.) using a deterministic mapping. Questions and answers are updated accordingly with word-boundary-aware replacement to avoid corrupting natural language phrases (e.g., the word "relationship" in "foreign key relationship" is preserved).

4 Baselines

Six baselines, same prompt, different context injection. All zero-shot. Each paradigm receives context natural to its architecture rather than enforcing token-budget parity, since the goal is to compare *paradigms*, not token counts.

Flat Text (FT). Dump the whole schema (tables, columns, FKs, lineage) as plain text. This mirrors what a developer does when pasting DDL into a long-context model.

Vector-RAG (VR). Embed schema elements with Sentence-BERT [15], retrieve k=15 chunks via FAISS [10], inject as context.

Graph-Augmented (GA). Extract a 3-hop BFS neighborhood around the tables mentioned in the question. For global questions with no anchor tables ("how many components?"), supply the full graph instead.

Tool-Use (TU). Hand the model nine graph algorithm tools: `get_neighbors`, `shortest_path`, `get_connected_component`, `get_lineage_chain`, `list_tables`. Up to 3 calls per question.

ReAct-Code (RC). The model writes and runs Python/NetworkX code in a sandboxed REPL, up to 5 rounds. More flexible than TU and serves as a budget ablation (5 code rounds vs. TU's 3 tool calls).

Oracle. Gold algorithmic output injected into the prompt. This establishes the performance ceiling.

For **Tier 2** we adapt four baselines: FT_{v2} (DDL-only), GA_{v2} (schema + value samples), TU_{v2} (interactive data tools), and RC_{v2} (free-form Python with value data access). RC_{v2} matches TU_{v2} on synthetic data (53%) but trails up to 20 pp on complex real-world schemas, consistent with structured tool APIs scaling better to real-world ETL complexity.

Table 3: Tier 1 results (EM % ± 95% CI, pooled across 5 datasets, 2000 bootstrap resamples). **Bold** = best non-oracle.

Model		FT	VR	GA	TU	RC
Gem.	Micro	76.6	72.9	75.4	**89.3**	81.4
	Macro	77.0	72.4	77.1	**86.6**	69.9
DS	Micro	69.5	71.0	76.8	**90.4**	79.4
	Macro	68.2	62.4	69.5	**88.4**	71.6
Qw.	Micro	63.2	69.5	80.9	**87.5**	64.6
	Macro	64.8	62.8	77.5	**82.5**	62.2
Oracle			99.5 / 99.8 / 100.0			

Table 4: EM (%) by difficulty. TU achieves near-perfect easy scores but plateaus on hard questions alongside static baselines.

Model	Diff.	FT	VR	GA	TU	RC
Gem.	Easy	82.7	82.5	82.2	**99.6**	90.4
	Med.	78.9	73.0	76.3	**90.3**	78.7
	Hard	56.1	46.5	55.6	59.9	**60.9**
DS	Easy	82.7	84.4	87.1	**98.0**	88.8
	Med.	65.3	65.3	76.0	**97.0**	76.3
	Hard	40.1	43.6	50.0	**60.4**	58.9
Qw.	Easy	77.4	84.0	86.6	**96.1**	72.2
	Med.	57.7	63.0	90.7	**92.0**	64.7
	Hard	33.2	40.1	51.0	**57.4**	44.1
Oracle Hard			99.0 / 99.5 / 100.0			

5 Experiments

5.1 Setup

We evaluate with **Gemini 2.5 Flash** [19] (closed frontier), **DeepSeek-V3** [2] (671B MoE, open-weight), and **Qwen2.5-72B** [14] (72B dense, open-weight), using greedy decoding (τ=0) for deterministic reproducibility. We report pooled **Micro-EM** (overall exact match) and **Macro-EM** (mean of per-subtype EM, weighting all 13 subtypes equally). For list-typed topology tasks (e.g., connected component membership), we apply **target-node normalization** (stripping the queried table from both prediction and gold prior to scoring) ensuring models are not penalized for harmless self-inclusion. An **Oracle** baseline injects gold algorithmic outputs, establishing an upper bound ($\geq$99.5% EM across all three models; the residual gap reflects minor formatting, not reasoning failures).

5.2 Tier 1: Main Results

Finding 1: Agentic baselines dominate. TU tops Tier 1 at 87–90% micro-EM, roughly 7–14 pp ahead of the best static baseline. RC also clears static methods through code generation.

Finding 2: Triviality Illusion. FT looks competitive at 76.6% micro-EM, but the illusion comes from `join_path` (33% of questions) inflating the average. Macro-EM, which weights subtypes equally, puts GA ahead of FT among static methods; all still trail TU by >8 pp (Appendix Figure 4).

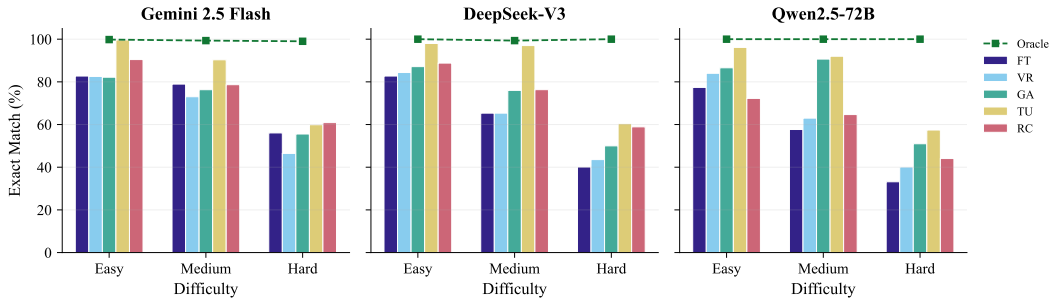

Figure 1: EM by difficulty level. All baselines plateau on hard questions while Oracle (dashed) achieves 100%.

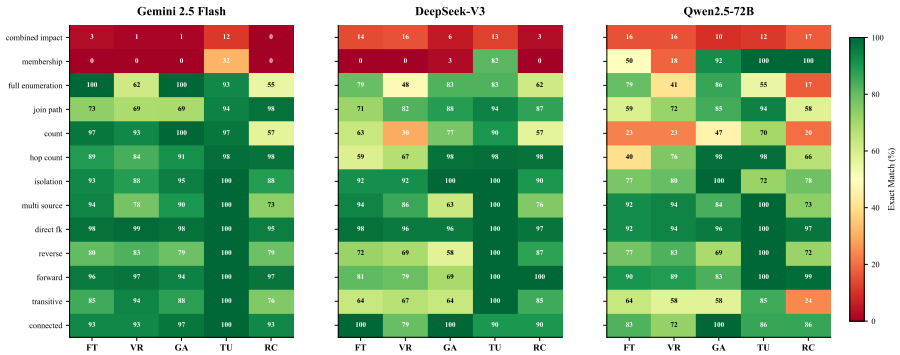

Figure 2: Tier 1 EM (%) by subtype for all three models. With target-node normalization, TU now averages 71% on `membership` (with Qwen reaching 100%), indicating that topology enumeration is fully solvable with tools. `combined_impact` remains at 0–17%, isolating compositional multi-hop reasoning as the core bottleneck.

5.3 Results by Difficulty

Easy/Medium patterns. Gemini FT scores 78.9% on Medium versus 82.7% on Easy. Medium is dominated by `join_path` (342 of 544 Medium questions), which has high FT accuracy since short FK paths are often present in the flat schema text. Easy includes subtypes like `count` (number of connected components) that require global graph access unavailable to FT. The distribution reflects subtype composition, not a flaw in difficulty labels.

5.4 Per-Subtype Analysis

Finding 3: Hard-task ceiling. All baselines plateau at ∼60% on hard questions while Oracle achieves >98%. The `combined_impact` subtype ($n = 69$) averages only 12% EM even with TU, identifying compositional graph reasoning as the fundamental bottleneck. RC (5 code rounds) shows a similar ceiling to TU (3 tool calls), suggesting the bottleneck is compositional reasoning itself rather than interaction budget.

5.5 Obfuscation

TU shows minimal degradation ($\Delta = -3.4\%$ Gemini, -4.2% DeepSeek, -3.7% Qwen) while static baselines lose 9–32pp. Obfuscation replaces table names only; column names are preserved, as all baselines operate on schema-level graph topology where column semantics play no role. RC shows near-zero penalty for DeepSeek ($\Delta = +0.6\%$), confirming that code execution against the graph object is largely invariant to name perturbation.

Table 5: Obfuscation penalty (4 datasets). TU is nearly invariant.

Model		FT	VR	GA	TU	RC
Gem.	Orig.	71.8	69.3	71.2	**91.2**	87.1
	Δ	−15.0	−25.8	−27.7	**−3.4**	−7.6
DS	Orig.	68.4	74.5	82.3	**91.4**	83.4
	Δ	−27.9	−31.9	−24.6	**−4.2**	+0.6
Qw.	Orig.	59.6	70.5	82.8	**88.4**	62.8
	Δ	−8.9	−13.8	−19.7	**−3.7**	−7.5

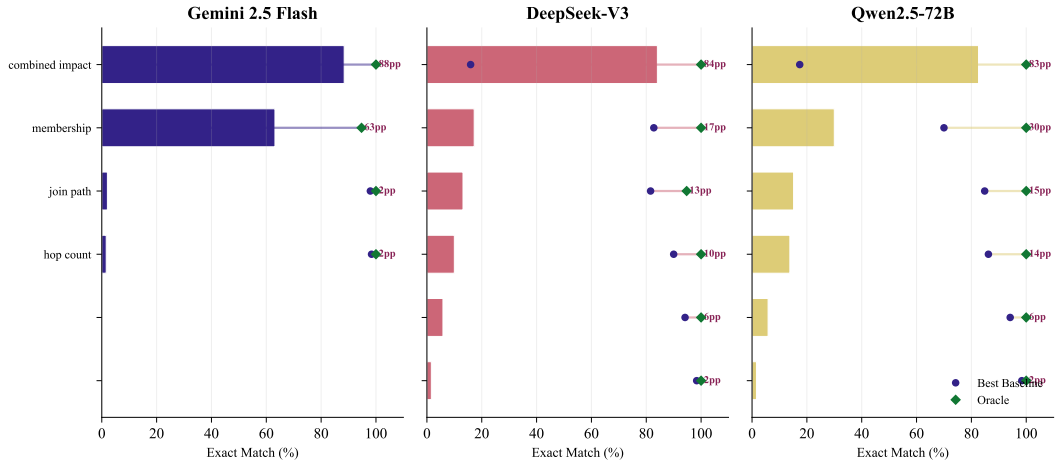

Figure 3: Unsolved subtypes: Oracle EM minus best non-oracle baseline. `combined_impact` retains>83 pp gap; `count`, `full_enumeration`, and `transitive` show 5–25 pp gaps. The remaining 9 of 13 subtypes are solved (<pp).

5.6 Tier 2: Value-Level Evaluation

Tier 2 tests value-level questions requiring interactive row-level data access (953 questions, 8 subtypes, AdventureWorks + Syn-Logistics). TU_{v2} achieves 59% EM (Gemini) and 48% (DeepSeek) versus 26–31% for static baselines. Both models confirm the same pattern: tools are **essential** (`cascade_count` averages 75% with tools vs. 0% without, reaching 100% on some schemas; both models) yet **insufficient** (`multi_hop_trace`: 27% both models). DeepSeek's lower TU score is driven by `row_provenance` (11% vs. 92%)—it identifies correct source tables but fails to parse structured row-level tool output. Full results in Appendix.

5.7 Discussion

Our results decompose graph reasoning into **retrieval-limited** subtypes (solved by tools) and **reasoning-limited** subtypes (unsolved even with tools). The ∼35% of questions that remain unsolved by both TU and GA largely overlap: their union improves EM by only 6pp, suggesting the bottleneck is compositional structural reasoning itself—the capability that GNN encoders are designed to provide.

`combined_impact` **specification.** Impact is the union of lineage-downstream tables and their FK-reachable neighbors: lineage transitive closure then FK BFS. This two-phase composition drives the 12% TU ceiling. RC receives 5 code-execution rounds (vs. TU's 3 tool calls) yet scores 0% (Gemini) on this subtype, ruling out tool budget as the primary bottleneck; the failure is compositional planning.

GA obfuscation penalty. The large GA Δ (−24 to −28 pp) reflects two compounding factors:

(1) loss of semantic priors (`DimCustomer`→`Table_K`) and (2) 45% of questions exceed the 3-hop radius, so obfuscation removes the cues that help LLMs infer structure beyond the retrieved fragment.

Tool tautology. TU achieving near-perfect scores on easy subtypes is by design: it isolates hard failures as genuine reasoning problems. TU hits 100% on `membership` but still only 12–13% on `combined_impact`. The ceiling is not tool availability; it is compositional multi-step reasoning.

5.8 Limitations

We evaluate three frontier LLMs with deterministic single-pass evaluation; broader model coverage (e.g., reasoning-optimized variants) would strengthen generality claims. Tier 2 covers one synthetic and one real-world dataset; extending to all five schemas is future work.

Responsible use. This benchmark is intended for scientific evaluation of LLM capabilities and limitations. Researchers should not apply automated agentic tools or sandboxed code execution to unauthorized or production environments without appropriate safeguards.

Dataset licenses. TPC-DS and TPC-DI are available under the TPC EULA (free for research). OMOP CDM uses the Apache 2.0 license. AdventureWorks is released under the Microsoft Public License. Syn-Logistics is our creation, released under MIT.

6 Conclusion

DW-Bench tests whether LLMs can reason about the graph topology of data warehouse schemas. We evaluate six baselines across five datasets using three models, yielding 1,046 schema-level and 953 value-level questions. Four findings stand out.

(1) Tool-use dominance. Give the LLM deterministic graph tools and it reaches 87–90% micro-EM, beating the best static baseline by 7–14 pp. LLMs can orchestrate algorithms they cannot internally simulate.

(2) Triviality illusion. Easy lookup subtypes inflate micro-EM. Switch to macro-EM (equal subtype weight) and the ranking shifts: what looked like widespread success reveals significant performance disparities.

(3) Hard-task ceiling. No baseline, including agentic code-generation, exceeds 61% on hard questions; the Oracle hits >99%. One subtype, `combined_impact` (12% EM with tools), accounts for most of the gap: it requires chaining lineage traversal with FK expansion, a compositional step that current baselines handle poorly.

(4) Obfuscation invariance. Tool-Use barely moves under obfuscation ($\Delta \leq 4$ pp). Static baselines drop 9–32 pp, consistent with tools operating on topology rather than memorized table names.

The natural next step is a **hybrid architecture**: pair interactive tools with learned graph representations (GNN encoders, graph-aware controllers) to close the compositional reasoning gap. DW-Bench supplies the per-subtype diagnostics needed to measure progress.

References

[1] Peter Baile Chen et al. Beaver: An enterprise benchmark for text-to-sql. In *ACL*, 2024.

[2] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[3] Darren Edge, Ha Trinh, Newman Cheng, et al. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[4] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. Talk like a graph: Encoding graphs for large language models. In *ICLR*, 2024.

[5] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with pytorch geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[6] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. SciPy Conference, 2008.

[7] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *NeurIPS*, 2024.

[8] George Hripcsak et al. Observational health data sciences and informatics (OHDSI): Opportunities for observational researchers. *Studies in Health Technology and Informatics*, 216, 2015.

[9] Bowen Jin, Chulin Xie, Jiawei Zhang, Kashob Kumar Roy Meng, Han Zhang, Shuiwang Zhang, Defu Bo, et al. Graph chain-of-thought: Augmenting large language models by reasoning on graphs. *Findings of ACL*, 2024.

[10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.

[11] Fangyu Lei et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In *ICLR*, 2025.

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[13] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chang, Fei Si, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In *NeurIPS*, 2023.

[14] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[15] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *EMNLP*, 2019.

[16] Jiashuo Sun, Chengjin Xu, Luminao Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *ICLR*, 2024.

[17] Jianheng Tang et al. Grapharena: Benchmarking large language models on graph computational problems. In *ICLR*, 2025.

[18] Zijian Tang, Ying Zhang, Sibo Cai, and Ruoxuan Wang. Llm-fk: Multi-agent llm reasoning for foreign key detection in large-scale complex databases. *arXiv preprint arXiv:2603.07278*, 2026.

[19] Gemini Team et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.

[20] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. Nlgraph: Can llms solve graph problems in natural language? In *NeurIPS*, 2023.

[21] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *EMNLP*, 2018.

[22] Zike Yuan et al. Gracore: Benchmarking graph comprehension and complex reasoning in large language models. In *COLING*, 2025.

[23] Yilun Zhao, Chen Zhao, Linyong Nan, Zhenting Qi, Wenlin Zhang, Xiangru Tang, Boyu Mi, and Dragomir Radev. Robut: A systematic study of table qa robustness against human-annotated adversarial perturbations. In *ACL*, 2023.

A Supplementary Figures

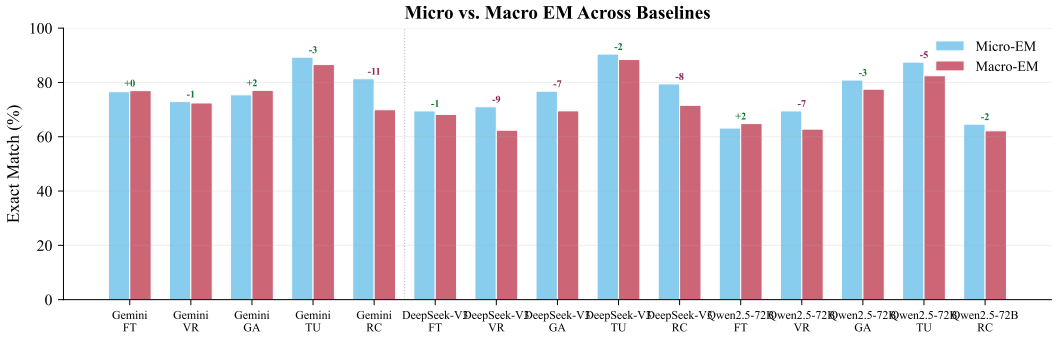

Figure 4: The Triviality Illusion: Micro vs. Macro EM. Negative deltas reveal that easy subtypes inflate aggregate scores for static baselines.

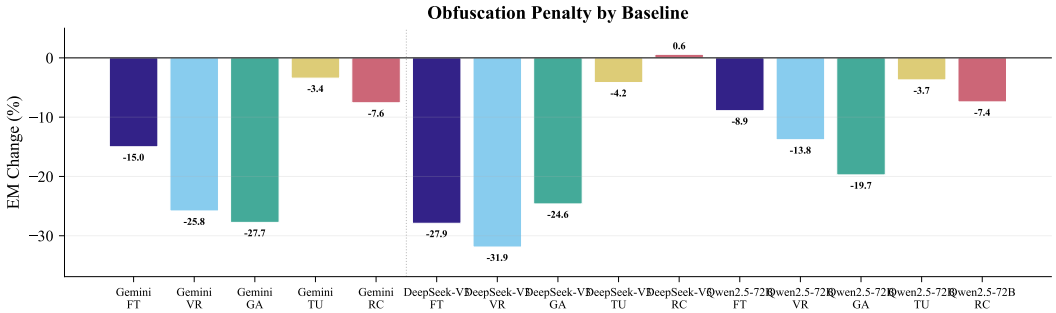

Figure 5: Obfuscation penalty by baseline and model. Tool-Use loses only 3–4% while static baselines lose 9–32%.

B Tier 2 Full Results

Table 6: Tier 2 value-level results (EM %, Syn-Logistics).

Subtype	n	FT_{v2}	GA_{v2}	TU_{v2}
cascade_count	58	0	0	**100**
row_provenance	84	0	8	**92**
row_impact	56	27	27	**38**
value_origin	60	48	**53**	52
multi_hop_trace	60	**30**	30	27
value_propagation	40	**85**	75	62
cross_silo_reach.	40	**58**	58	52
shared_source	35	**37**	26	23
Overall	433	30	31	**59**

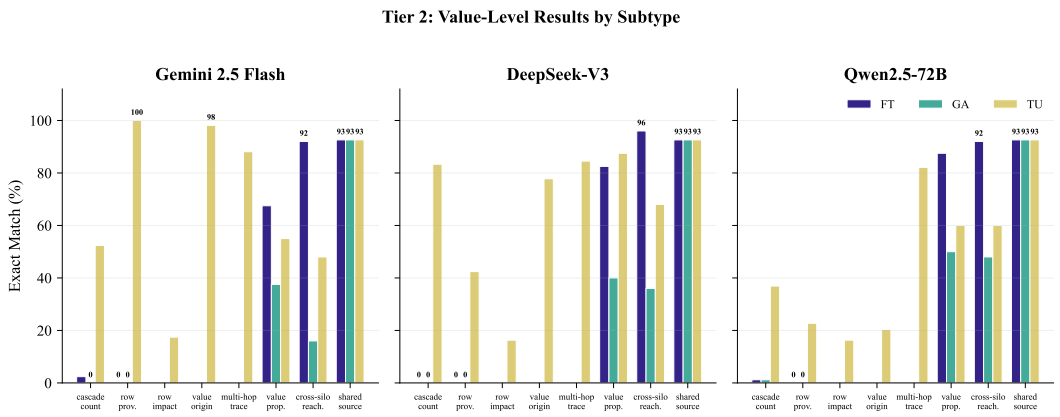

Figure 6: Tier 2 EM by subtype. `cascade_count` and `row_provenance` require tools (100%/92% vs. 0%).

C Hop-Distance Analysis

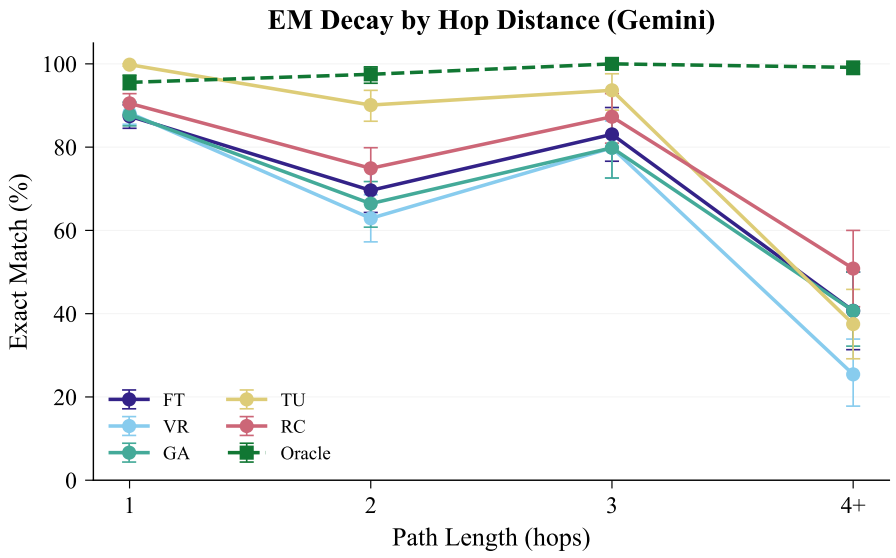

Figure 7: EM by gold path length. Static baselines collapse beyond 3 hops; Tool-Use degrades more gracefully.

Table 7: Minimum evidence hops per subtype (Syn-Logistics). 45.4% of questions exceed the GA 3-hop BFS radius.

Subtype	n	Exceeds 3h	%	GA EM
count	26	26	100%	100%
membership	20	20	100%	0%
combined_impact	34	34	100%	0%
full_enum	25	25	100%	100%
connected	25	25	100%	100%
isolation	25	25	100%	100%
transitive	29	4	14%	93%
hop_count	44	5	11%	100%
direct_fk	36	3	8%	100%
join_path	26	2	8%	81%
multi_source	42	0	0%	95%
forward	20	0	0%	100%
reverse	20	0	0%	100%
Total	372	169	**45.4%**	83.1%

D Dataset Details

Syn-Logistics Design. The Syn-Logistics dataset guarantees $n \geq 20$ per subtype via three parallel 4-hop lineage chains across five components. Table names are drawn from realistic supply-chain terminology (64 tables, 96 FK, 35 lineage).

Lineage Edge Curation. **AW**: Microsoft reference docs (39 edges). **TPC-DS**: No lineage (pure FK). **TPC-DI**: Official spec ETL flows (21 edges). **OMOP**: OHDSI ETL conventions (21 edges). **Syn-Log**: By construction (35 edges).

E Evaluation Details

Models. **Gemini 2.5 Flash** (Google, closed-weight, accessed via API) and **DeepSeek-V3** (DeepSeek, 671B MoE, MIT licensed, accessed via API). Both use greedy decoding (τ=0, p=1.0) with a single deterministic pass per question for exact reproducibility.

Scoring. *Exact Match (EM)*: predicted answer must exactly match the gold answer after canonicalization. *Path validation*: for `join_path`, alternative shortest paths of equal length are validated against the FK adjacency matrix. *List canonicalization*: lists are lowercased and sorted alphabetically before comparison; order does not matter. *F1*: token-level overlap between predicted and gold answer sets.

Retrieval. Vector-RAG uses `all-MiniLM-L6-v2` [15] embeddings with FAISS exact search and k=15 retrieved chunks. Graph-Augmented uses 3-hop BFS neighborhoods from mentioned tables. Tool-Use provides 9 graph algorithm tools with up to 3 calls per question. ReAct-Code provides a Python REPL with NetworkX and allows up to 5 code execution rounds per question.

Statistical treatment. All figures include 95% bootstrap confidence intervals computed via 2,000 resamples over questions. Per-question results (JSON) are released for community bootstrap and stratification analyses.

F Example Questions and Predictions

Table 8: Representative questions across difficulty levels and categories, with gold answers and Tool-Use (TU) predictions. ✓ = correct, ✗ = incorrect. Tool-Use succeeds on lookup-style questions (direct FK, shortest path, cascade count) but fails on exhaustive enumeration (membership) and multi-hop composition (combined impact, provenance tracing).

Subtype	**Question**	**Gold**	**TU Pred.**
`direct_fk` *Easy*	Is there a direct FK between `DimProduct` and `FactInternetSales`?	yes	yes ✓
`join_path` *Medium*	Shortest FK path from `DimCustomer` to `DimProduct`?	[DimCustomer, FactInternetSales, DimProduct]	[DimCustomer, FactInternetSales, DimProduct] ✓
`membership` *Hard*	Which component contains `DimCurrency`? List all tables.	[DimCurrency, FactCurrencyRate, DimDate, . . .] (12)	[DimCurrency] ✗
`combined_impact` *Hard*	Which tables are transitively affected (lineage+FK) if `raw_purchase_orders` is modified?	[stg_purchase_orders, dim_supplier, . . .] (8)	[stg_purchase_orders] ✗
`cascade_count` *Tier 2*	How many downstream rows affected if we delete `shipment_id=SH042` from `raw_shipments`?	147	147 ✓
`multi_hop_trace` *Tier 2*	Trace provenance of `order_id=ORD-2891` in `mart_profit_analysis` to raw sources.	[raw_purchase_orders → stg_orders → dim_product → mart_profit_analysis]	[raw_purchase_orders, mart_profit_analysis] ✗